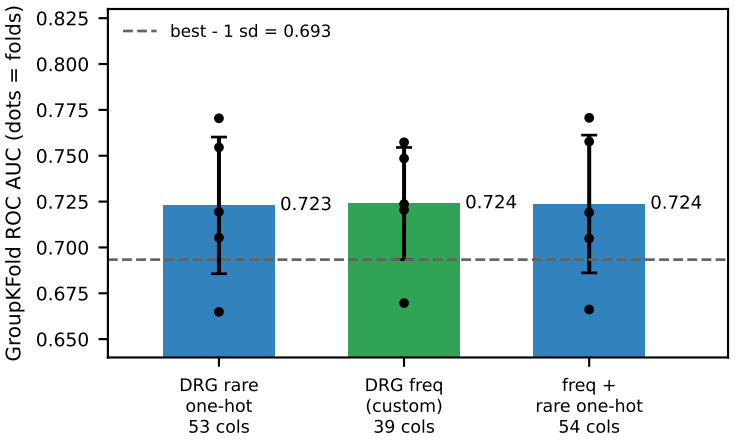
Day 30 Cheat Sheet: Custom Transformers and One Full Pipeline

Your own fit/transform steps, inside one scikit-learn Pipeline, from raw table to prediction

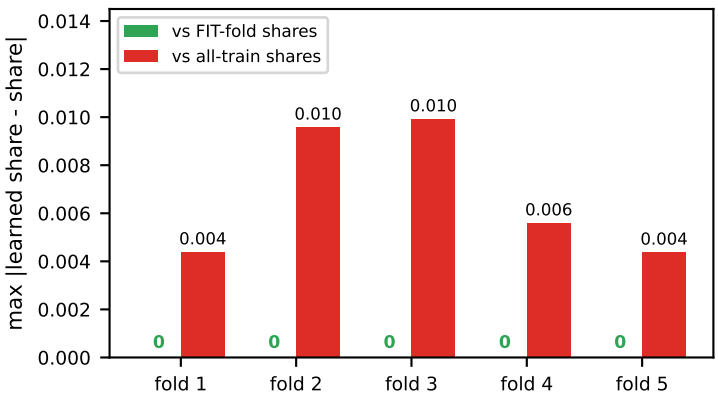
```
ANATOMY OF A CUSTOM TRANSFORMER
class X(TransformerMixin, BaseEstimator)
__init__(self, min_count=1): store params only
fit(self, X, y=None): learn state, return self
transform(self, X): use ONLY the learned state
get_feature_names_out()
FunctionTransformer(func)
A class
.set_output(transform="pandas")
check_estimator(MyTransformer())
```

```
RULE
mixin LEFT of BaseEstimator (check_estimator checks it)
no logic, no data; get_params / clone rely on it
learned attributes end in "_" (freq_, cap_)
never re-learn from the batch you are scoring
names flow through to coefficients and set_output
stateless steps: log1p, 'I50.9' -> 'I' (named func, not lambda)
anything that must LEARN in fit: frequencies, caps, rates
every step returns a DataFrame with column names
~45 API checks; on_fail=None returns a list
```

Same pipeline, three DRG options (green = champion)



Leak check: FrequencyEncoder learns ONLY the fit fold



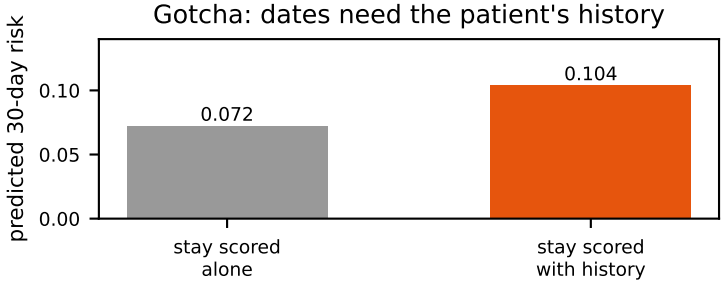
THIS DATA: 2,655 simulated stays from 1,000 patients (seed 30), readmit rate 17.8%, 100 DRG codes, split by patient: train 1,997, test 658. The pipeline takes the 12 RAW columns and builds 54 features with freq + rare one-hot (39 with freq only).
5-fold GroupKFold ROC AUC: DRG rare one-hot 0.723 | DRG FrequencyEncoder 0.724 | both 0.724 (fold sd about 0.04).
Rule: best minus 1 sd = 0.693; simplest that qualifies = freq (39 columns). Test once: AUC 0.743.
Leak check: in all 5 folds the encoder's shares match the fit fold exactly (max diff 0) and differ from all-train by up to 0.0099.
check_estimator(FrequencyEncoder()): 45 passed, 0 failed. joblib reload gives identical predictions: True.

RULES OF THUMB

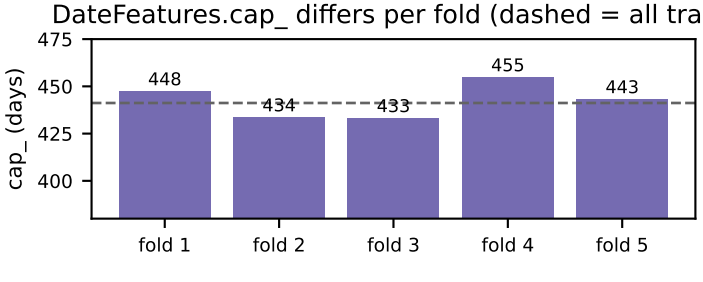
- If a step learns from data, it lives inside the Pipeline, so GroupKFold refits it on every fit fold.
- Unseen codes at predict time must not crash: map them to 0 (or an 'other' level) on purpose.
- Keep custom classes in a .py module. joblib (and MLflow tomorrow) pickles a reference to the class, not its code.
- Batch-dependent features (days since last visit) need the patient's history rows at scoring time.

```python
class FrequencyEncoder(TransformerMixin, BaseEstimator):
    def __init__(self, min_count=1):
        self.min_count = min_count
    def fit(self, X, y=None):
        X = validate_data(self, X, dtype=None, ensure_all_finite=False)
        self.freq_ = [pd.Series(c).value_counts(normalize=True).to_dict() for c in X.T]
        return self
    def transform(self, X):
        X = validate_data(self, X, dtype=None, ensure_all_finite=False, reset=False)
        return np.column_stack([pd.Series(c).map(f).fillna(0.0) for c, f in zip(X.T, self.freq_)])
```

Gotcha: dates need the patient's history



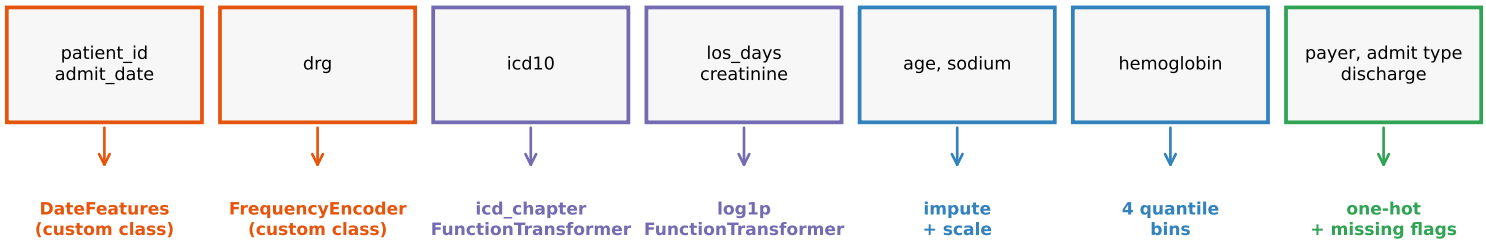
DateFeatures.cap_ differs per fold (dashed = all train)



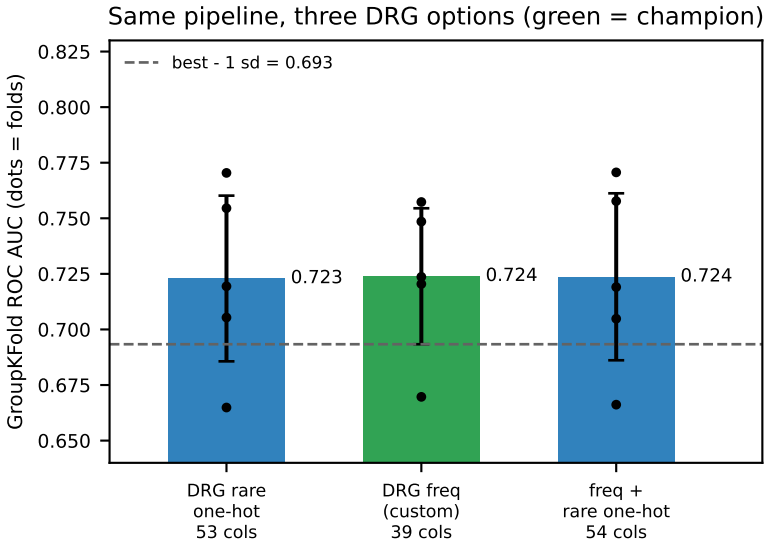
Infographic: One pipeline, raw table to prediction

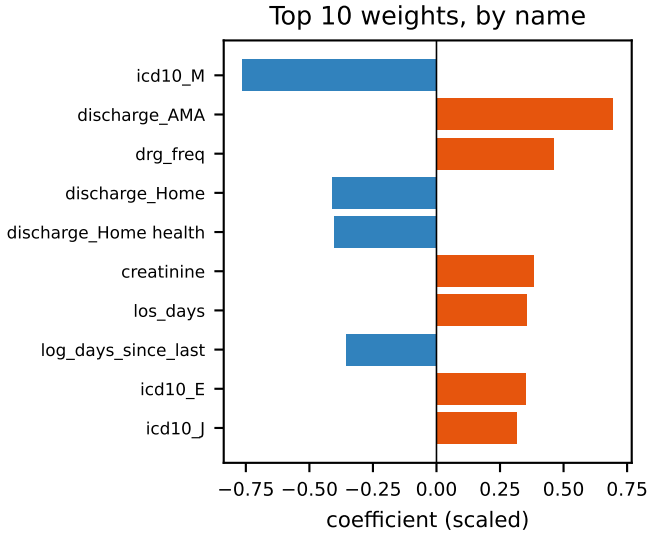
30-day readmission, simulated healthcare data, custom steps that learn only on the fit fold

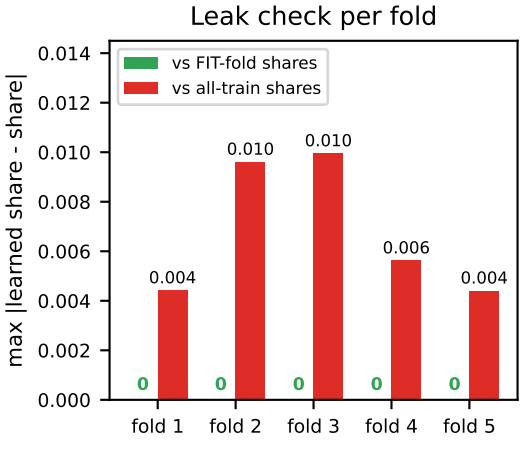
RAW COLUMNS IN	FEATURES BUILT	CV ROC AUC	TEST AUC
12	39	0.724	0.743

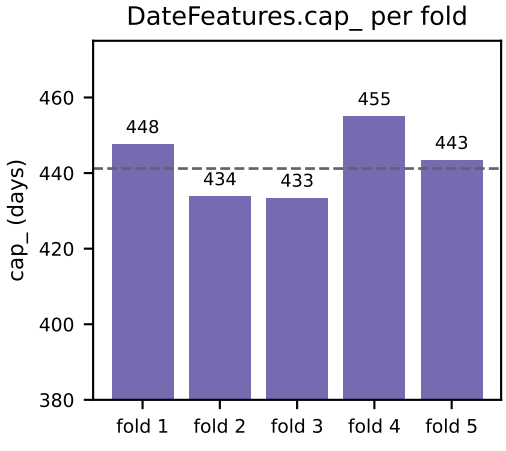


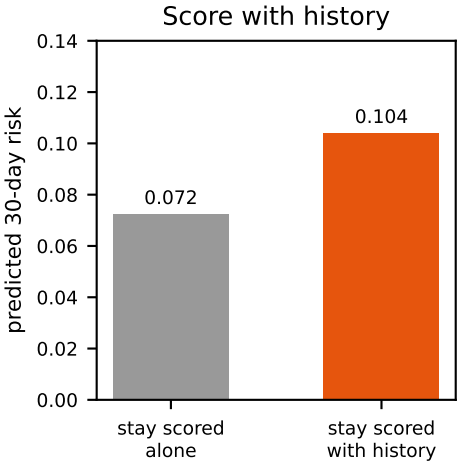
ColumnTransformer -> LogisticRegression -> risk -> joblib.dump -> MLflow (Day 31)











How to read it

1. Custom steps are just transformers: the FrequencyEncoder alone (0.724, 39 cols) ties DRG one-hot (0.723, 53 cols); simplest wins.
2. Green zeros: in every fold the learned DRG shares equal the fit fold's shares, not the full training set's. No leakage.
3. DateFeatures learns a cap per fold (433 to 455 days); first visits get the cap plus a first_visit flag.
4. One stay scored alone looks like a first visit (0.072); with the patient's earlier stay it is 0.104. Send history at scoring time.

Hands-On Worksheet: ds-genai-90-days / day30

60-90 minutes | day30/readmit_transformers.py + day30/readmit_pipeline.ipynb + README.md

[ ] mkdir day30, conda activate ds90, open the folder in VS Code

[ ] Simulate 2,655 raw stays from 1,000 patients, seed 30; split by patient (snippet 1)

[ ] Write readmit_transformers.py: FrequencyEncoder, DateFeatures, icd_chapter (snippet 2)

[ ] Explain in a markdown cell: FunctionTransformer vs a class (what must be LEARNED?)

[ ] Build the ONE pipeline with build() and set_output(transform='pandas') (snippet 3)

[ ] Print the DataFrame columns: find dates__* and drg_freq__drg_freq

[ ] Write your selection rule BEFORE racing: best CV AUC, simplest within 1 sd

[ ] Race rare / freq / freq+rare with GroupKFold by patient (snippet 4)

[ ] Leak check: learned DRG shares == fit-fold shares in all 5 folds?

[ ] check_estimator(FrequencyEncoder(), on_fail=None): how many passed? (snippet 5)

[ ] Refit the champion on TRAIN, open the test set ONCE

[ ] joblib.dump, reload, predict 3 raw rows; score a return stay alone vs with history

[ ] README: pipeline diagram, CV table, leak check, history gotcha, limits (simulated)

[ ] git add day30 && git commit && git push (snippet 6)

[ ] Stretch: frequency table in Snowflake SQL or a Snowpark DataFrame (snippet 7)

```
pipe = build("freq")                                  # raw columns in
cv = cross_validate(pipe, train, ytr, groups=gtr, cv=GroupKFold(5), return_estimator=True)
fe = cv["estimator"][0]["prep"].named_transformers_["drg_freq"][0]   # fitted on fold 1 only
joblib.dump(pipe.fit(train, ytr), "readmit_pipeline.joblib")         # MLflow-ready tomorrow
```

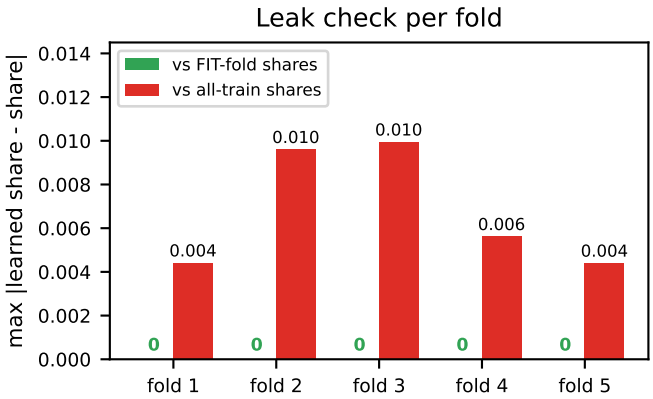
Raw columns: _____ Features built: _____

CV AUC rare: _____ freq: _____ both: _____

Champion: _____ Test AUC: _____

check_estimator passed: _____ Risk alone/with history: _____ / _____

Notes: _____



Day 30 Self-Check Quiz

1. When is FunctionTransformer enough, and when do you need a class with fit and transform?

_____

2. Why must __init__ only store parameters, and why do learned attributes end with an underscore?

_____

3. What does the FrequencyEncoder do with a DRG code it never saw during fit?

_____

4. How do you prove a custom transformer learned only from the fit fold inside GroupKFold?

_____

5. check_estimator complained about mixin order. What was wrong and how do you fix it?

_____

6. Why save the custom classes in a .py file instead of defining them in the notebook?

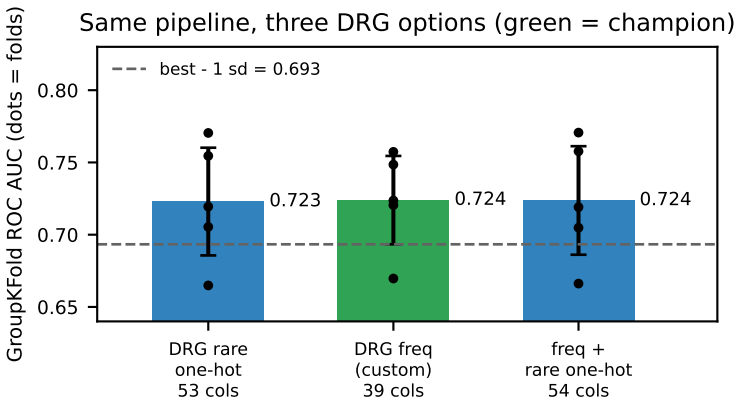
_____

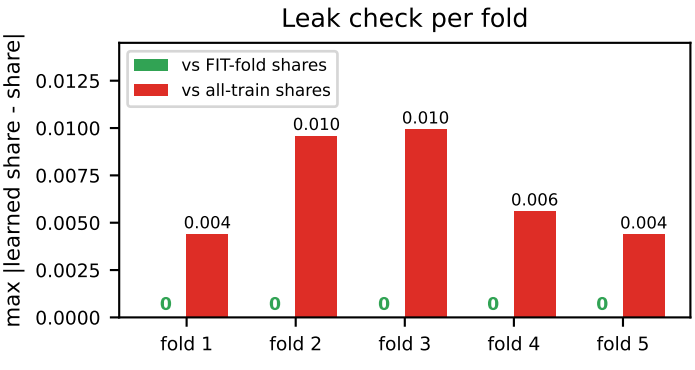
7. A return stay scored alone got a lower risk than with its history. Why?

_____

8. Why use a lambda-free, named function inside FunctionTransformer?

_____





Answers:
1 FunctionTransformer for stateless math (log1p, ICD chapter); a class when something must be learned in fit (DRG shares, the date cap)
2 clone() and get_params rebuild the step from __init__ params for every fold; the "_" marks state learned in fit, which check_is_fitted looks for
3 maps it to 0 (a share of zero), so new codes next month do not crash the pipeline; 4 validation codes across folds hit this path
4 return_estimator=True, pull each fold's fitted encoder, compare freq_ with that fold's shares: max diff 0, vs up to 0.0099 against all train
5 BaseEstimator was listed first; write class FrequencyEncoder(TransformerMixin, BaseEstimator). After tags for strings and NaN: 45 passed, 0 failed
6 joblib and MLflow pickle a reference to the class; loading needs the same importable module (MLflow: code_paths)
7 alone it looks like a first visit (cap + first_visit flag), 0.072 vs 0.104; send the patient's prior stays, or compute the gap in SQL with LAG
8 lambdas cannot be pickled by the standard pickler, so joblib.dump of the pipeline would fail; a named module-level function can